CSA0603-DESIGN AND ANALYSIS OF ALGORITHM

CO 2- ASSESSMENT 3 – DEBUGGING AND OPTIMIZATION TASK

NAME:S.SRI MATHI

REG NO:192424095

QUESTION

Q1. Debugging Closest Pair Code (Comparison with Wrong Variable)

The following brute force Closest Pair implementation calculates distances correctly but compares the newly computed distance against the wrong reference value. Carefully analyze the update logic and identify why the minimum pair is not always preserved. Apply debugging to ensure that the smallest distance found so far is maintained correctly throughout the nested loops. Optimize the implementation by avoiding repeated tuple indexing where possible. Analyze the time complexity of the corrected algorithm and rewrite the final version with proper comments.

```
import math
def closest_pair(points):
  min_dist = float('inf')
  for i in range(len(points)):
     for j in range(i+1, len(points)):
        x1, y1 = points[i]
        x2, y2 = points[j]
        dist = math.sqrt((x1-x2)**2 + (y1-y2)**2)
        if dist < i:
           min_dist = dist
  return min_dist
```

DEBUGGING

Step 1: Run the given code

Observation:

The program runs without displaying an error, but **no output is produced**.

Reason identified:

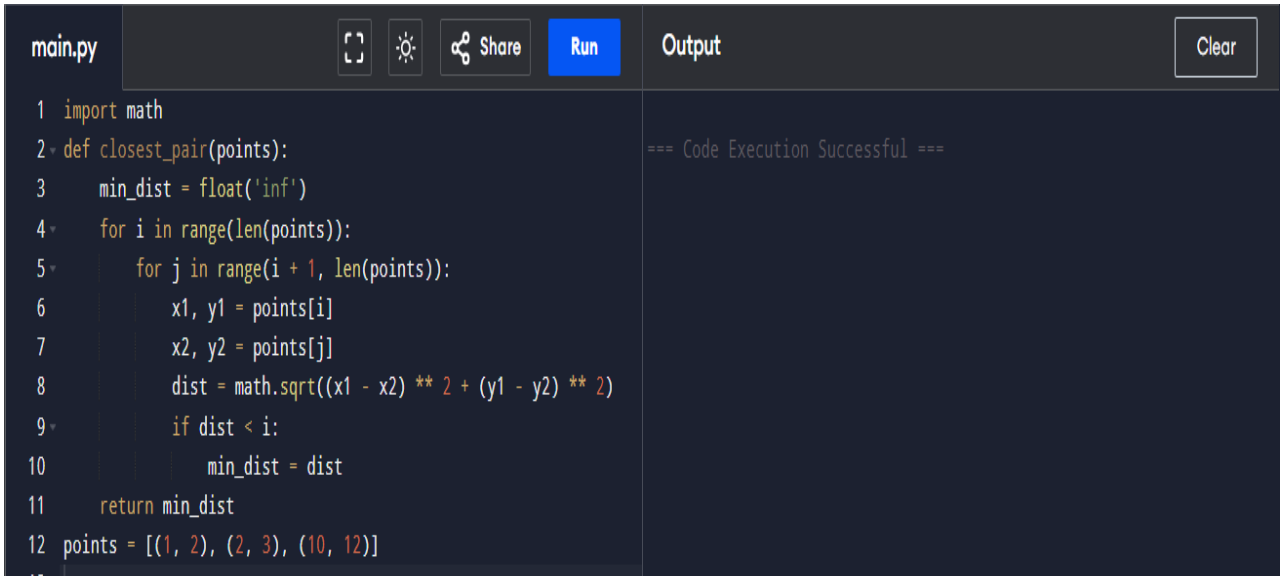
The code only defines the function def closest_pair(points):

The function is not called, and there is no print() statement.

Step 2 : Add input

Adding these lines below the function

points = [(1, 2), (2, 3), (10, 12)]



Observation:

The answer should not be infinity because distances exist between the given points.

The distance between (1, 2) and (2, 3) is approximately 1.414

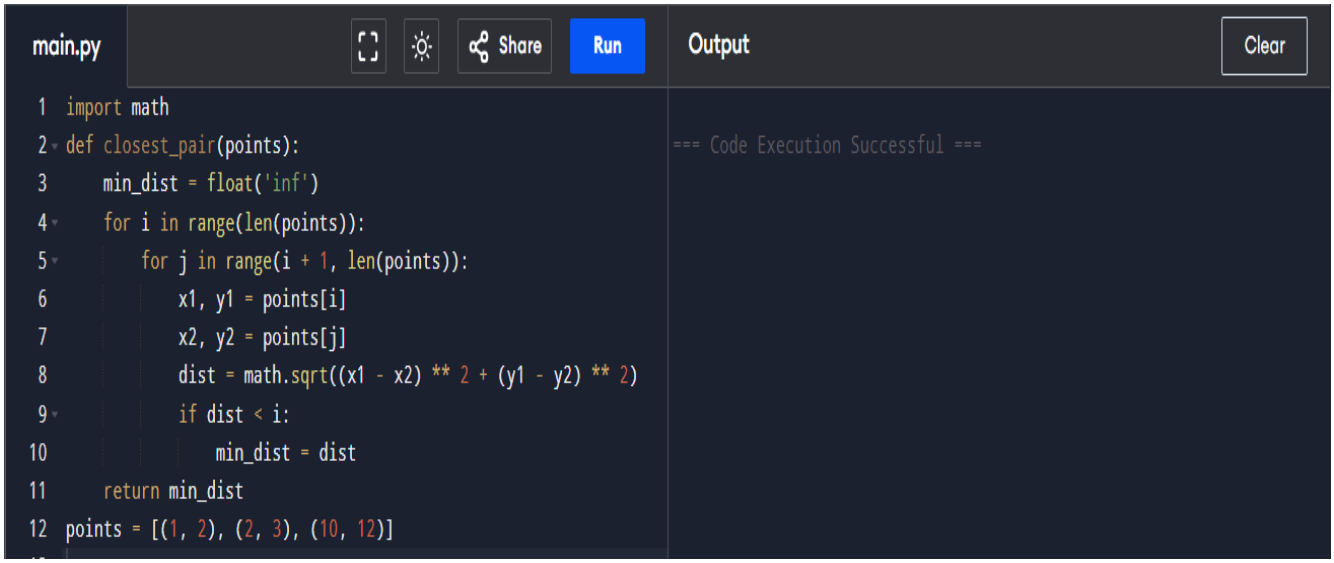
Therefore, getting inf shows that min_dist was never updated.

STEP 3: Add function call

Add this line

result = closest_pair(points)

print("Minimum distance:", result)



Step 4: Locate the logical error

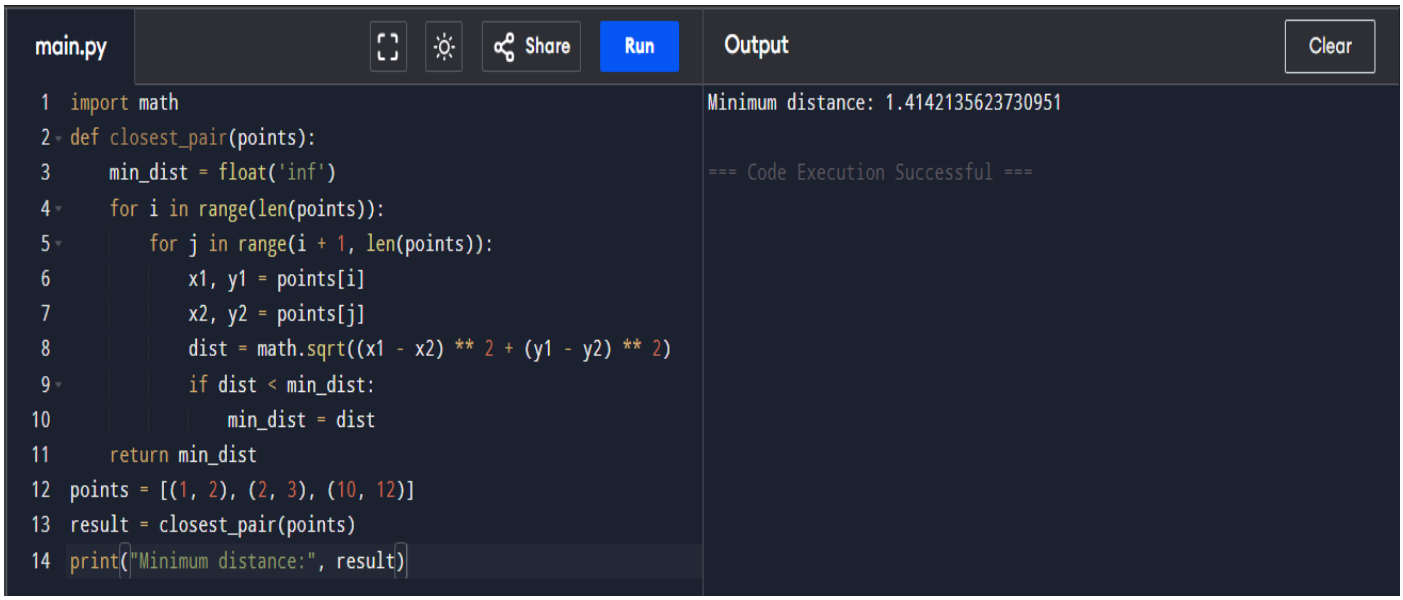
The incorrect line is: if dist < i

Comparing a distance with a point index is logically wrong.

The new distance must be compared with the minimum distance found so far.

Step 5: Correct the comparison line

Replace: if dist < i With if dist < min_dist :



Observation

The program now produces the correct minimum distance.

Step 6: Optimize repeated tuple indexing

In the current code x1, y1 = points[i] is placed inside the inner loop.

Move outside the inner loop

```
for i in range(len(points)):
    x1, y1 = points[i]
    for j in range(i + 1, len(points)):
        x2, y2 = points[j]
```

main.py Share Run Output Clear

```python
def closest_pair(points):
    min_dist = float('inf')
    for i in range(len(points) - 1):
        x1, y1 = points[i]
        for j in range(i + 1, len(points)):
            x2, y2 = points[j]
            dist = math.sqrt(
                (x1 - x2) ** 2 + (y1 - y2) ** 2
            )
            if dist < min_dist:
                min_dist = dist
    return min_dist
points = [(1, 2), (2, 3), (10, 12)]
result = closest_pair(points)
print("Minimum distance:", result)
```

```
Minimum distance: 1.4142135623730951

=== Code Execution Successful ===
```

Algorithm: Brute-Force Closest Pair

Input: A list of points (x, y)
Output: The minimum distance between any two points

- Start.
- Read the list of points.
- Set min_dist to infinity.
- Select each point one by one using index i.
- Store the coordinates of the first point as x1 and y1.
- Compare it with every point after it using index j.
- Store the second point coordinates as x2 and y2.
- Calculate the Euclidean distance:
- distance = $\sqrt{((x1 - x2)^2 + (y1 - y2)^2)}$
- Compare the calculated distance with min_dist.
- If the new distance is smaller, update min_dist.
- Repeat until all unique pairs are checked.
- Display min_dist.
- Stop.

PSEUDO CODE:

```
ALGORITHM ClosestPair(points)
    min_dist ← infinity
    n ← length(points)
    FOR i ← 0 TO n - 2 DO
        x1 ← points[i].x
        y1 ← points[i].y
        FOR j ← i + 1 TO n - 1 DO
            x2 ← points[j].x
            y2 ← points[j].y
            dist ← √((x1 - x2)² + (y1 - y2)²)
            IF dist < min_dist THEN
                min_dist ← dist
            END IF
        END FOR
    END FOR
    RETURN min_dist
END ALGORITHM
```

Time complexity :$O(n^2)$